

**INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO**

INGENIERÍA EN SISTEMAS COMPUTACIONALES
UNIDAD DE APRENDIZAJE: BASES DE DATOS

**FUNDAMENTOS TEÓRICOS, CONCEPTUALES Y
ARQUITECTURA DE LOS SISTEMAS GESTORES DE BASES DE
DATOS**

Monografía Analítica de la Unidad Temática I

Alumno: Emmanuel Morales González

Tema: Unidad Temática I – Introducción y Fundamentos de Bases de Datos

INSTITUTO POLITÉCNICO NACIONAL



ESCOM

1. Dato, Información y Base de Datos

En la ciencia de la computación y la ingeniería de software, la distinción rigurosa entre dato, información y base de datos constituye la piedra angular sobre la que descansa toda la teoría de la gestión de información. Un dato representa una representación simbólica (numérica, alfabética, algorítmica, espacial o booleana) de un atributo, hecho o entidad aislada del mundo real, desprovisto de contexto evaluativo intrínseco o significado operativo per se (Silberschatz et al., 2020). Por sí mismo, un dato aislado carece de capacidad para orientar la acción humana o computacional. Por su parte, la información surge cuando un conjunto estructurado de datos es sometido a procesos contextuales de filtrado, ordenamiento, agrupación, agregación y procesamiento semántico, transformándose en un mensaje con significado explícito capaz de reducir la incertidumbre y fundamentar la toma de decisiones racionales en las organizaciones (Elmasri & Navathe, 2016).

A partir de este continuo cognoscitivo, el concepto de base de datos formaliza la preservación estructurada de dichos elementos. Para comprender su naturaleza desde la literatura científica autorizada, se analizan dos definiciones de autores fundamentales en la disciplina:

- **Definición según Elmasri y Navathe (2016):** Los autores definen una base de datos como una colección lógicamente coherente de datos relacionados, que posee un significado implícito intrínseco y que representa algún aspecto del mundo real (denominado minimundo o Universo de Discurso, UoD). Elmasri y Navathe enfatizan que una colección aleatoria de datos no puede ser considerada genuinamente una base de datos; para serlo, debe ser diseñada, construida y poblada con un propósito específico predeterminado y para satisfacer las necesidades de un grupo definido de usuarios y aplicaciones.
- **Definición según Silberschatz, Korth y Sudarshan (2020):** Por otro lado, estos autores definen una base de datos como una colección estructurada de datos interrelacionados gestionada con el propósito primario de proporcionar un entorno que sea a la vez conveniente, eficiente y seguro para recuperar y almacenar información masiva. Su perspectiva enfatiza fuertemente el balance operativo entre las restricciones de abstracción de datos, la gestión transparente del hardware físico y la consistencia matemática de los datos frente a transacciones concurrentes y fallos imprevistos.
- **Definición industrial según Oracle Corporation (2024):** En concordancia con los autores académicos citados, Oracle sintetiza que una base de datos es una recopilación organizada de información o datos estructurados que normalmente se almacenan electrónicamente en un sistema informático, controlada comúnmente por un sistema de gestión de bases de datos (DBMS). Oracle subraya la evolución contemporánea hacia modelos relacionales, bases de datos orientadas a documentos y arquitecturas en la nube desacopladas.

Al contrastar estas formulaciones, se evidencia una complementariedad dialéctica: mientras Elmasri y Navathe (2016) abordan la base de datos desde una perspectiva ontológica y semántica (la fidelidad con la que el modelo conceptual replica el minimundo real), Silberschatz et al.

(2020) y Oracle (2024) adoptan un enfoque pragmático y sistémico centrado en la eficiencia computacional, la robustez transaccional y la capacidad operativa del software gestor.

2. Características Fundamentales de una Base de Datos

A diferencia de un repositorio pasivo de archivos, una base de datos administrada por un SGBD exhibe un conjunto de propiedades sistémicas interdependientes que aseguran su integridad, disponibilidad y rendimiento (Date, 2004; Ramakrishnan & Gehrke, 2003):

- **Integración:** La base de datos se concibe como una entidad única y unificada que amalgama datos que anteriormente residían de forma fragmentada en múltiples aplicaciones aisladas. Cada ítem de información se interconecta lógicamente con otros mediante relaciones formales (e.g., claves primarias y foráneas), eliminando silos informativos.
- **Persistencia:** Garantiza que el estado de los datos sobreviva indefinidamente a la terminación de los procesos de software que los crearon o modificaron. Los datos residen en medios de almacenamiento secundario no volátil (discos magnéticos, arreglos SSD NVMe o almacenamiento en la nube), garantizando su disponibilidad continua.
- **Redundancia controlada:** A diferencia de los sistemas de archivos donde los mismos campos se duplican incontroladamente en múltiples departamentos, en una base de datos cada hecho se almacena, idealmente, en un único lugar lógico. Cuando deliberadamente se introduce redundancia física por motivos de rendimiento y optimización de lecturas, el SGBD se encarga de propagar automáticamente las actualizaciones para evitar divergencias lógicas (Elmasri & Navathe, 2016).
- **Integridad:** Consiste en el aseguramiento de la exactitud, validez y coherencia de los datos mediante la imposición rigurosa de reglas de negocio, tipos de datos, restricciones de entidad (claves primarias no nulas y únicas), integridad referencial (claves foráneas consistentes) y restricciones semánticas de dominio (Date, 2004).
- **Independencia de datos:** Es la capacidad de modificar el esquema en un nivel de abstracción sin alterar los esquemas de los niveles superiores. Comprende la independencia física (modificar estructuras de disco e índices sin cambiar el esquema conceptual) y la independencia lógica (modificar tablas o vistas sin alterar los programas de usuario).
- **Seguridad:** Mecanismo integral que protege los datos contra accesos no autorizados, modificaciones ilícitas o destrucción maliciosa. Opera mediante autenticación, autorización granular basada en roles (RBAC), control discrecional y obligatorio de acceso, y cifrado de datos tanto en reposo como en tránsito (Silberschatz et al., 2020).
- **Concurrencia:** Capacidad del motor para permitir que miles de transacciones y usuarios interactúen y modifiquen simultáneamente la base de datos sin interferir entre sí. Se logra mediante protocolos de control de concurrencia como el bloqueo en dos fases (2PL) o el control de concurrencia multiversión (MVCC), previniendo anomalías clásicas como lecturas sucias, actualizaciones perdidas o lecturas no repetibles (Ramakrishnan & Gehrke, 2003).
- **Recuperación:** Conjunto de algoritmos y mecanismos diseñados para restaurar la base de datos a un estado consistente y libre de corrupción tras una falla imprevista (caídas de

hardware, apagones, errores de red o excepciones lógicas de software), apoyándose fundamentalmente en el registro de transacciones WAL (Härder & Reuter, 1983).

3. Archivos Frente a Bases de Datos: Génesis Histórica de los SGBD

Durante las primeras décadas de la computación comercial (1950-1960), las organizaciones gestionaban sus datos mediante el enfoque tradicional de procesamiento de archivos provisto por el sistema operativo. En este paradigma, cada aplicación departamental (e.g., Nómina, Facturación, Inventarios) definía, creaba y manipulaba sus propios archivos físicos (planos, secuenciales o indexados). Conforme las corporaciones crecieron en escala y complejidad, este esquema reveló graves limitaciones inherentes que condujeron a una crisis en el mantenimiento del software (Elmasri & Navathe, 2016):

- **Redundancia incontrolada de datos:** Dado que cada departamento desarrollaba sus propios programas de forma aislada, idénticos datos (por ejemplo, el nombre, la dirección y el número de seguro social de un cliente o empleado) se duplicaban en docenas de archivos independientes. Esto multiplicaba de manera exponencial los costos de almacenamiento físico en disco.
- **Inconsistencia de datos:** Como consecuencia directa de la redundancia no supervisada, cuando un cliente cambiaba de domicilio o un producto modificaba su precio unitario, la actualización se reflejaba únicamente en el archivo del departamento que ejecutaba la transacción inmediata, dejando obsoletos los registros de los demás sistemas. Ello provocaba estados contradictorios de la verdad donde diferentes reportes empresariales arrojaban cifras disímiles e irreconciliables (Date, 2004).
- **Dependencia estricta programa-datos:** En los sistemas de archivos, la descripción física de la estructura de los datos (la longitud exacta de cada campo, el orden posicional de los bytes, los delimitadores y el tipo de codificación) se codificaba rígidamente dentro del código fuente de cada programa en lenguajes como COBOL, Fortran o C. En consecuencia, si se requería ampliar el campo del código postal o añadir un nuevo atributo, resultaba obligatorio modificar, recompilar, probar y redespigar cada uno de los cientos de programas que abrían dicho archivo, encareciendo el mantenimiento de software a niveles insostenibles.
- **Dificultad de acceso a nuevos datos y aislamiento de información:** La generación de consultas ad-hoc imprevistas requería escribir un programa completamente nuevo para recorrer archivos heterogéneos, sin posibilidad de ejecutar consultas declarativas dinámicas. Asimismo, la falta de mecanismos nativos de concurrencia y seguridad delegaba en cada programador la pesada tarea de inventar soluciones artesanales de bloqueo y protección de datos, propiciando violaciones de integridad y accesos concurrentes destructivos (Silberschatz et al., 2020).

Esta confluencia de factores patológicos en los sistemas de archivos propició el nacimiento formal de los Sistemas Gestores de Bases de Datos (SGBD). El SGBD emergió como una capa de software intermedio especializada que sustrae la administración y el almacenamiento de datos del ámbito privado de cada aplicación, centralizando el catálogo conceptual de metadatos,

garantizando la independencia lógica y física, y proveyendo servicios unificados de concurrencia, integridad, seguridad y optimización declarativa de consultas (Codd, 1970; Tsichritzis & Klug, 1978).

4. Usuarios de una Base de Datos: Roles, Responsabilidades y Permisos

El ecosistema de una base de datos corporativa involucra diversos perfiles humanos con distintas competencias técnicas, requerimientos funcionales y privilegios operativos sobre los recursos del sistema (Elmasri & Navathe, 2016; Ramakrishnan & Gehrke, 2003):

- **Diseñador de la Base de Datos (Database Designer):** Responsable de identificar y formalizar el minimundo de la organización. Sus funciones primordiales consisten en interactuar con los grupos de interés para relevar requerimientos, construir el esquema conceptual (mediante modelos como Entidad-Relación extendido), transformarlo en un esquema lógico estructurado (normalización relacional hasta 3FN/FNBC), y definir las restricciones de integridad y directrices de particionamiento físico inicial. En cuanto a permisos, cuenta con facultades de definición de esquemas (DDL) en entornos de desarrollo y homologación, pero carece de permisos de alteración y manipulación directa en ambientes productivos.
- **Administrador de la Base de Datos (Database Administrator - DBA):** Es la máxima autoridad técnica y operativa sobre el sistema gestor. Sus responsabilidades comprenden la instalación, configuración y afinación del SGBD, la gestión del almacenamiento físico en disco, la definición y asignación de esquemas de seguridad y autenticación, la monitorización continua del rendimiento, la ejecución de políticas de respaldo (backups fríos y calientes), la recuperación ante desastres y la aplicación de parches de actualización. El DBA posee los más altos privilegios administrativos (rol SUPERUSER, SYSDBA), permitiéndole ejecutar cualquier comando DDL, DML y DCL, otorgar o revocar permisos a otros usuarios y reconfigurar parámetros del kernel del gestor.
- **Programador de Aplicaciones (Application Developer):** Profesional de software que implementa las interfaces de usuario, la lógica de negocio y las aplicaciones transaccionales que interactúan con la base de datos a través de librerías, conectores y APIs (e.g., JDBC, ODBC, ORMs). Su rol exige formular consultas DML eficientes, parametrizadas y libres de inyecciones SQL, invocar procedimientos almacenados y gestionar los límites transaccionales (COMMIT y ROLLBACK). Sus permisos están restringidos a la ejecución de sentencias DML (SELECT, INSERT, UPDATE, DELETE) y ejecución de procedimientos sobre objetos específicos, careciendo de privilegios para alterar la estructura de las tablas o modificar políticas de seguridad del sistema.
- **Usuario Final (End User):** Persona que accede a la base de datos de manera directa o a través de interfaces de aplicación para satisfacer sus tareas laborales cotidianas. Se clasifican en: (a) Usuarios casuales o analistas, que utilizan herramientas analíticas (BI) o formuladores de consultas SQL visuales para consultar reportes e indicadores; (b) Usuarios ingenuos o paramétricos, que constituyen el volumen mayoritario (cajeros, operadores de inventario,

clientes de plataformas web), interactuando únicamente con formularios predefinidos que ejecutan transacciones fijas programadas; y (c) Usuarios sofisticados (científicos de datos e ingenieros), que explotan capacidades analíticas avanzadas. Sus permisos son estrictamente de solo lectura o acotados mediante roles discretos con privilegios mínimos a través de vistas.

5. Ciclo de Vida de una Base de Datos: Fases y Entregables Formales

El ciclo de vida del desarrollo de una base de datos (Database Life Cycle - DBLC) constituye un proceso de ingeniería sistemático, disciplinado y metodológico diseñado para garantizar que el sistema cumpla con los requerimientos operativos, de rendimiento y de seguridad de la organización (Elmasri & Navathe, 2016; Connolly & Begg, 2015). A continuación, se describen sus fases secuenciales y sus entregables tangibles:

- **Fase 1: Planificación y Análisis de Requerimientos:** Se delimita el alcance del sistema, se identifican las reglas del negocio, las fuentes de datos y las restricciones del minimundo mediante entrevistas, cuestionarios y análisis de documentación existente. Entregable: Documento de Especificación de Requerimientos de Software y Datos (SRS), glosario de términos del negocio y matriz de trazabilidad de necesidades.
- **Fase 2: Diseño Conceptual:** Modela la realidad del negocio con total independencia del motor SGBD específico y del hardware. Se identifican entidades, atributos, dominios, relaciones, cardinalidades y restricciones estructurales. Entregable: Diagrama Entidad-Relación Extendido (EER) formalmente validado junto con el Diccionario de Datos Conceptual.
- **Fase 3: Diseño Lógico:** Transforma el modelo conceptual al modelo de datos soportado por el tipo de SGBD elegido (comúnmente el modelo relacional). Incluye la aplicación exhaustiva de técnicas de normalización (Primera, Segunda, Tercera Forma Normal y FNBC) para erradicar redundancias indeseadas y anomalías de actualización. Entregable: Esquema Lógico Relacional (conjunto de esquemas de relación con claves primarias y foráneas debidamente especificadas) y matriz de dependencias funcionales.
- **Fase 4: Diseño Físico:** Adapta el esquema lógico a las características y particularidades técnicas del motor gestor específico seleccionado (e.g., PostgreSQL, Oracle, SQL Server). Se seleccionan las estructuras de almacenamiento interno, los métodos de acceso (árboles B+, hash), la creación de índices primarios y secundarios, el particionamiento de tablas y las directrices de seguridad física. Entregable: Scripts DDL de creación de la base de datos (Data Definition Language) completamente comentados y plan de dimensionamiento de almacenamiento.
- **Fase 5: Implementación y Carga de Datos:** Ejecución de los scripts DDL en los servidores, compilación de vistas, disparadores (triggers) y procedimientos almacenados. Se ejecutan los procesos ETL (Extracción, Transformación y Carga) para migrar los datos históricos o iniciales provenientes de sistemas legados. Entregable: Instancia funcional de la base de datos poblada, scripts ETL automatizados y bitácoras de auditoría de migración sin inconsistencias.

- **Fase 6: Pruebas y Validación:** Sometimiento del sistema a rigurosas baterías de pruebas de integridad referencial, pruebas de carga y estrés transaccional, pruebas de regresión, evaluación de planes de ejecución de consultas (EXPLAIN ANALYZE) y verificación de protocolos de respaldo y recuperación ante desastres. Entregable: Informe formal de Aseguramiento de Calidad (QA), métricas de latencia/rendimiento y acta de aceptación operativa.
- **Fase 7: Operación, Monitorización y Mantenimiento:** Puesta en producción del sistema. Comprende la monitorización continua del consumo de recursos de memoria y CPU, la reindexación periódica, la reorganización de datos fragmentados, la resolución de peticiones de cambios y la afinación continua del rendimiento. Entregable: Reportes periódicos de desempeño (SLA), bitácoras de respaldo y manuales de administración operativa actualizados.

6. El Sistema Gestor de Bases de Datos (SGBD / DBMS)

Un Sistema Gestor de Bases de Datos (SGBD o DBMS) es un paquete de software complejo que interactúa con las aplicaciones de usuario, el compilador del lenguaje y el sistema operativo para facilitar la definición, construcción, manipulación y compartición controlada de bases de datos entre múltiples usuarios (Silberschatz et al., 2020).

Módulos Componentes de un SGBD

Arquitecturalmente, un SGBD moderno se divide en dos grandes subsistemas funcionales: el procesador de consultas y el gestor de almacenamiento (Silberschatz et al., 2020):

- **Procesador de Consultas (Query Processor):** Contiene el intérprete de DDL (procesa definiciones y las registra en el catálogo/diccionario de datos), el compilador y parser de DML (analiza sintáctica y semánticamente las sentencias SQL, traduciéndolas a expresiones de álgebra relacional), el motor de ejecución de consultas (ejecuta el plan de bajo nivel interpretado) y, de forma crítica, el Optimizador de Consultas (Query Optimizer), el cual analiza múltiples planes de evaluación alternativos y selecciona el camino con menor costo computacional estimado en términos de accesos I/O a disco y consumo de CPU, fundamentado en estadísticas internas (Selinger et al., 1979).
- **Gestor de Almacenamiento (Storage Manager):** Capa responsable de mediar entre los datos lógicos de bajo nivel y el subsistema de archivos del sistema operativo. Sus submódulos clave son: (a) Gestor de memoria intermedia (Buffer Pool Manager), que administra la memoria RAM manteniendo páginas de datos activas y ejecutando políticas de reemplazo (e.g., LRU); (b) Gestor de archivos y métodos de acceso, que organiza la estructura física de registros y páginas en disco; (c) Gestor de transacciones, que vela por el cumplimiento estricto de las propiedades ACID; (d) Gestor de concurrencia, que aplica políticas de bloqueo o marcas de tiempo; y (e) Gestor de recuperación (Recovery Manager), que escribe de manera síncrona el registro WAL antes de que las modificaciones alcancen el disco físico (Härder & Reuter, 1983).

Lenguajes de Bases de Datos

El lenguaje SQL unifica funcionalmente múltiples sublenguajes especializados orientados a interactuar con las diversas facetas del SGBD (Date, 2004):

- **Data Definition Language (DDL):** Permite definir y modificar las estructuras de esquemas, relaciones, dominios e índices. Comandos fundamentales: CREATE, ALTER, DROP, TRUNCATE. Cada sentencia DDL actualiza de forma automática el Diccionario de Datos o catálogo del sistema.
- **Data Manipulation Language (DML):** Permite a los usuarios acceder, consultar y manipular los datos contenidos en las relaciones. Puede ser declarativo (e.g., SQL clásico, donde se especifica qué datos se desean obtener sin indicar el procedimiento algorítmico) o procedimental (donde se guía el flujo paso a paso). Comandos esenciales: SELECT, INSERT, UPDATE, DELETE.
- **Data Control Language (DCL):** Permite administrar la seguridad y el control de privilegios sobre los objetos de la base de datos. Comandos principales: GRANT (concede privilegios a usuarios o roles) y REVOKE (retira privilegios previamente asignados).
- **Control de Transacciones (TCL):** Permite gestionar los límites y la atomicidad de las operaciones transaccionales. Comandos principales: COMMIT (consolida y hace permanentes las alteraciones en disco), ROLLBACK (aborta la transacción y restaura el estado original) y SAVEPOINT (establece puntos de control intermedios para cancelaciones parciales).

Clasificación de los SGBD

Los sistemas gestores se categorizan bajo múltiples criterios taxonómicos (Elmasri & Navathe, 2016):

- **Por el modelo de datos subyacente:** Relacionales (PostgreSQL, Oracle, MySQL), Orientados a Documentos (MongoDB), Clave-Valor (Redis), Orientados a Columnas (Cassandra), Grafos (Neo4j) y Híbridos/Multimodelo.
- **Por el número de usuarios:** Monousuario (SQLite, Microsoft Access) y Multiusuario (DB2, PostgreSQL, Oracle Enterprise).
- **Por la distribución del sistema:** Centralizados (almacenados y procesados en una sola máquina host) y Distribuidos (cómputo y almacenamiento fragmentado o replicado en clústeres mediante red).
- **Por el modelo de licenciamiento:** Código abierto (PostgreSQL, MariaDB) y Propietarios comerciales (Oracle Database, Microsoft SQL Server).

7. Arquitectura del Sistema de Base de Datos

La Arquitectura de Tres Niveles ANSI/SPARC

Formalizada por el comité ANSI/X3/SPARC a finales de los años setenta (Tsichritzis & Klug, 1978), la arquitectura de tres niveles persigue como objetivo primordial separar

nítidamente las aplicaciones de usuario de las complejidades físicas del hardware de almacenamiento, garantizando la independencia de datos:

- **1. Nivel Externo o de Vistas:** Es el nivel más próximo a los usuarios. Describe la parte de la base de datos relevante para cada usuario o grupo específico, ocultando el resto del sistema. Un sistema puede contener infinitas vistas externas personalizadas según roles y permisos.
- **2. Nivel Conceptual:** Representa la visión global comunitaria de toda la base de datos. Describe qué datos se almacenan, las entidades, relaciones, restricciones de integridad y reglas del negocio. Oculta deliberadamente todos los detalles de almacenamiento físico, actuando como el mediador central del sistema.
- **3. Nivel Interno o Físico:** Es el nivel más cercano al hardware. Describe la estructura de almacenamiento físico de la base de datos en disco: representación binaria de registros, organización de páginas, secuencias de bloques, asignación de memorias intermedias y estructuras de indexación (árboles B+, hash).

A través de esta arquitectura se consiguen dos modalidades cardinales de independencia:

- **Independencia Lógica de Datos:** Capacidad de modificar el esquema conceptual (añadir nuevas tablas, campos o relaciones) sin requerir reescrituras de los esquemas externos ni de los programas existentes.
- **Independencia Física de Datos:** Capacidad de reorganizar la capa interna (cambiar la configuración de índices, mover archivos a otros volúmenes o migrar a SSDs) sin alterar el esquema conceptual ni las aplicaciones.

Arquitecturas Centralizadas Frente a Cliente-Servidor

En una arquitectura centralizada, todos los componentes (el SGBD, los datos en disco, la lógica de aplicación y el software de interfaz) residen y se ejecutan en un único computador central (mainframe o minicomputador), al que los usuarios se conectaban históricamente mediante terminales tontas sin capacidad de cómputo local (Elmasri & Navathe, 2016).

Por contraste, la arquitectura Cliente-Servidor desacopla las responsabilidades computacionales aprovechando la proliferación de estaciones de trabajo y redes locales: el cliente gestiona la interfaz gráfica, la validación preliminar de datos y la experiencia de usuario, mientras que el servidor aloja el motor del SGBD, ejecuta la optimización y procesamiento de consultas SQL, y gestiona la persistencia, la concurrencia y la seguridad. Esta arquitectura evolucionó hacia modelos de tres capas (Cliente Web -> Servidor de Aplicaciones / Middleware -> Servidor de Base de Datos SGBD), permitiendo una escalabilidad horizontal y balanceo de carga sin precedentes.

8. Modelos de Datos: Definición, Importancia y Clasificación

Un modelo de datos es una colección integrada de conceptos, herramientas conceptuales y notaciones formales utilizadas para describir y estructurar datos, sus relaciones semánticas, su significado intrínseco y las restricciones de consistencia aplicables al sistema (Silberschatz et al.,

2020; Codd, 1970). Su importancia radica en que actúa como el lenguaje de comunicación unificado entre los expertos del negocio y los ingenieros de software, garantizando que la arquitectura computacional sea una representación fiel del dominio organizacional.

Clasificación por Nivel de Abstracción

- **Modelos Conceptuales o de Alto Nivel:** Enfocados en la percepción humana del mundo real. Emplean conceptos intuitivos como entidades, atributos y relaciones. Ejemplo cumbre: Modelo Entidad-Relación (Chen, 1976).
- **Modelos Lógicos o Representacionales:** Proporcionan conceptos comprensibles para los usuarios pero estructurados de forma que pueden ser mapeados directamente a la arquitectura de un SGBD comercial. Ejemplos: Modelo Relacional (tablas de Codd), Modelo en Red y Modelo Jerárquico.
- **Modelos Físicos o de Bajo Nivel:** Describen cómo se organizan los bytes, punteros, bloques y páginas de almacenamiento en los medios no volátiles de memoria secundaria.

Clasificación por Estructura

- **Modelo Jerárquico:** Estructura los datos en una jerarquía de árbol estricta donde cada registro hijo posee un único padre (relaciones 1:N). Rigidez severa ante relaciones M:N.
- **Modelo en Red (CODASYL):** Extensión del modelo jerárquico mediante conjuntos y punteros directos, permitiendo que un registro hijo tenga múltiples padres (relaciones complejas en grafo direccionado).
- **Modelo Relacional:** Formulado matemáticamente por Edgar F. Codd (1970) basado en la teoría de conjuntos y la lógica de predicados de primer orden. Los datos se perciben como tuplas en relaciones bidimensionales, prescindiendo de punteros físicos visibles para el usuario y manipulándose mediante álgebra relacional declarativa.
- **Modelo Orientado a Objetos y Objeto-Relacional:** Integra conceptos de la programación orientada a objetos (encapsulamiento, métodos, herencia y polimorfismo) con las capacidades de persistencia del SGBD (PostgreSQL).
- **Modelos NoSQL / Grafos / Multidimensionales:** Optimizados para volúmenes masivos, esquemas flexibles o recorridos topológicos directos (documentos JSON, grafos LPG o cubos OLAP).

9. Fronteras de la Investigación: Conexión con el Estado del Arte

Para trascender los cimientos fundacionales y comprender la evolución contemporánea de la Unidad Temática I, resulta indispensable articular las investigaciones científicas de vanguardia que redefinen la arquitectura de datos:

- **Optimización física e independencia de datos con Machine Learning:** Kraska et al. (2018) proponen los 'Learned Index Structures', demostrando que las estructuras de indexación tradicionales en el nivel interno (árboles B+) pueden reemplazarse por modelos estadísticos de regresión que aprenden la función de distribución acumulada (CDF) de las claves de

acceso, reduciendo el espacio de memoria en un 70% y aumentando el rendimiento de lectura. Ello ratifica la vigencia del principio de independencia física de ANSI/SPARC: el nivel interno se optimiza radicalmente sin impactar el esquema conceptual ni el código de las aplicaciones.

- **Límites estructurales de los modelos y adyacencia de grafos:** Özsu (2020) en su riguroso sondeo 'Graph Processing: A Survey' analiza las limitaciones intrínsecas del álgebra relacional de Codd (1970) al procesar entidades hiperconectadas. Demuestra formalmente que consultas recursivas y exploraciones topológicas sufren cuellos de botella por JOINs múltiples en motores tabulares, validando que modelos orientados a grafos basados en 'index-free adjacency' superan por varios órdenes de magnitud a los modelos relacionales en consultas de caminos y redes complejas.
- **Redefinición de la arquitectura cliente-servidor en la nube:** Verbitski et al. (2018) en 'Amazon Aurora' revolucionan la arquitectura de tres niveles en infraestructuras virtualizadas. Proponen el principio 'The Log is the Database', desacoplando estrictamente el cómputo (motor SQL y transacciones) del almacenamiento persistente distribuido. Al enviar únicamente registros atómicos del log WAL a una flota de almacenamiento distribuido multizona que asume asíncronamente la materialización de páginas de datos, multiplican por cinco el throughput transaccional y reducen a escasos segundos la recuperación de caídas (crash recovery).

Referencias

- Chen, P. P. S. (1976). The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36. <https://doi.org/10.1145/320434.320440>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Connolly, T. M., & Begg, C. E. (2015). *Database systems: A practical approach to design, implementation, and management* (6th ed.). Pearson Education.
- Date, C. J. (2004). *An introduction to database systems* (8th ed.). Addison-Wesley.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson Education.
- Härder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys*, 15(4), 287–317. <https://doi.org/10.1145/289.291>
- Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). The case for learned index structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)* (pp. 489–504). Association for Computing Machinery. <https://doi.org/10.1145/3183713.3196909>
- Oracle Corporation. (2024). *¿Qué es una base de datos?* Oracle México. <https://www.oracle.com/mx/database/what-is-database/>

- Özsu, M. T. (2020). Graph processing: A survey. *ACM Computing Surveys*, 53(2), Article 42, 1–38. <https://doi.org/10.1145/3382189>
- Ramakrishnan, R., & Gehrke, J. (2003). *Database management systems* (3rd ed.). McGraw-Hill.
- Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., & Price, T. G. (1979). Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data* (pp. 23–34). Association for Computing Machinery. <https://doi.org/10.1145/582095.582099>
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.
- Tsichritzis, D., & Klug, A. (1978). The ANSI/X3/SPARC DBMS framework report of the study group on database management systems. *Information Systems*, 3(3), 173–191. [https://doi.org/10.1016/0306-4379\(78\)90001-7](https://doi.org/10.1016/0306-4379(78)90001-7)
- Verbitski, A., Gupta, A., Saha, D., Brahmadesam, M., Gupta, K., Mittal, R., Krishnamurthy, S., Maurice, S., Kharatishvili, T., & Corey, J. (2018). Amazon Aurora: On avoiding distributed consensus for I/O, commits, and read replicas. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)* (pp. 589–596). Association for Computing Machinery. <https://doi.org/10.1145/3183713.3196937>